[Nam Le] C Programing Review

Session 2

Status: In progress...

Section: Linux System Programming

Topic: C Programming Language Review

Duration: 3 working days (8 hours/day)

1. C Fundamentals

```
#include <stdio.h>
int add(int a, int b) {
    return a + b;
}
void swap(int* a, int* b) {
    int* c = a;
    a = b;
    b = c;
    printf("%d va %d\n", *a, *b);
}
int main() {
    int x = 10;
    printf("%p\n", (void*)&x);
    int y = 11;
    printf("%p\n", (void*)&y);
    printf("Tong x + y = %d\n", add(x, y));
    printf("After swap, x va y = ");
    swap(&x, &y);
    return 0;
}
```

Inside main(), declare and assign values to variables x and y.



			x	y			
Value			10	11			
Address			0x108	0x104			

Vùng nhớ Stack trong OS

When add(x, y) is called, the program copies the values of x and y into parameters a and b of add() → A new Stack Frame is created for add().

(Stack Frame)			[main]		[add]		
			x	y	*Return Address*	a	b
Value			10	11			
Address			0x108	0x104		0x096	0x092

Vùng nhớ Stack trong OS

Copy values into a and b.

(Stack Frame)			[main]		[add]		
			x	y	*Return Address*	a	b
Value			10	11		10	11
Address			0x108	0x104		0x096	0x092

Vùng nhớ Stack trong OS

Return the result from add() to main() (calculated by CPU, stored in a register).

Return using the Return Address → Clean up the Stack Frame of add() → Read the Return Address to jump to the next instruction (swap).

Call swap() by passing the direct addresses of x and y (pass 0x108 and 0x104 to pointers a and b) → Create a new Stack Frame for swap().

(Stack Frame) [main] [swap]

			x	y	*Return Address*	Pointer a	Pointer b	Pointer c
Value			10	11				
Address			0x108	0x104		0x108	0x104	(temp)

Vùng nhớ Stack trong OS

```
int* c = a;
   a = b;
   b = c;
```

Pointer c copies the address stored in pointer a (gets address 0x108).

Pointer a changes its target address to 0x104, pointing to value 11 in main. Same logic applies to pointer b.

Unlike C++, even though they share the same stack frame address, pointer a (in swap) and variable x (in main) lie in different memory locations. In C++ (or other high-level languages), the same memory location can hold two or more objects at once → **no pass by Reference**, only **pass by Value**

IF WE DO NOT PASS THE ADDRESSES OF X AND Y TO SWAP()

Pointers only hold values while the function copies them → The original values will not change.

2. POINTER & ARRAY

```
#include <stdio.h>
#include <stdlib.h>
int find_max(int *arr, int size) {
    int max = *arr;
    for (int i = 1; i < size; i++) {
        if (*(arr + i) > max) {
            max = *(arr + i);
        }
    }
    return max;
}
int main() {
```

```
    int *arr = NULL;
    printf("Dia chi cua arr: %p\n", arr);
    int size = 5;
    arr = (int*) malloc(size * sizeof(int));
    printf("Nhap mang: ");
    for (int i = 0; i < size; i++) {
        scanf("%d", (arr + i));
    }
    printf("The address of element 0 & 1 is : %p %p\n", arr, arr + 1);
    printf("Max of Array = %d\n", find_max(arr, size));
    free(arr);
    arr = NULL;
}
```

Stack và Heap chia sẻ chung 1 không gian

		*arr					
Value							
Address		nil (NULL)					

Vùng nhớ Stack

		*arr					
Value		arr[0]	arr[1]	arr[2]	arr[3]	arr[4]	
Address		0x1000	0x1004	0x1008	0x100C	0x1010	

Vùng nhớ Heap

<——————————————————————>

20 byte

Pointer arr points to address 0x1000 (**the base address** of the array allocated dynamically on the Heap).

Input phase: arr + 1 = Base Address + (1 * data type size)

free(arr) & arr = NULL;

→ Memory is released + arr is reset from 0x1000 to 0 (NULL pointer) to prevent program crashes when accessing old addresses.

Pointer to Pointer

```
#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int data;
    struct Node *next;
} Node;

void push_front(Node **head, int value)
{
    Node *new_node = malloc(sizeof(Node));

    new_node->data = value;
    new_node->next = *head;

    *head = new_node;
}

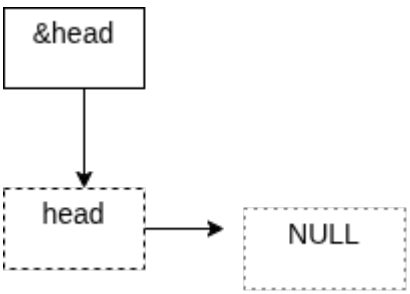
int main()
{
    Node *head = NULL;

    push_front(&head, 10);
    push_front(&head, 20);

    printf("%d -> %d\n", head->data, head->next->data);

    return 0;
}
```

call push_front(&head);



in function , Node **head →head is holding the address of head (main)

Then: `*head = New_node; → Directly update head in main`

IF CHANGE **Node **head** TO **Node *head** → Pointer head only receives a copy of head's value from main, so it cannot modify the original head.

→ Compile error (*head = new_node) because you cannot dereference when head is already pointing to a node value.

3. Dynamic Memory

malloc , calloc, realloc

malloc (memory allocation)

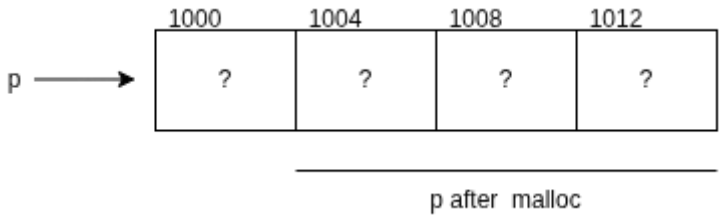
Allocates memory by specified bytes. Returns a pointer to the first byte.

Initial values are garbage values or NULL.

Syntax: `*ptr = malloc(byte-size);`

EXAMPLE:

```
int *p = malloc(4 * sizeof(int));
```



calloc (Contiguous Allocation):

Allocates memory for multiple contiguous blocks (e.g., creating an array).

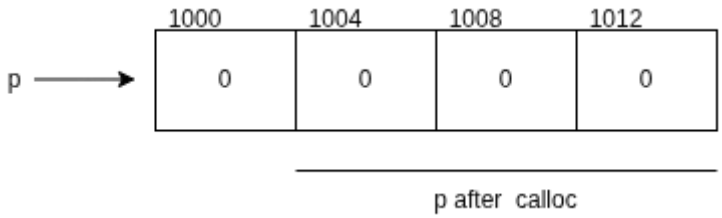
Automatically initializes all allocated bytes to 0.

Syntax:

```
*ptr = cmalloc(n, element-size);
```

EXAMPLE:

```
int *p = calloc(4, sizeof(int));
```



realloc (Re-allocation):

Changes (expands or shrinks) the size of previously allocated memory from malloc or calloc.

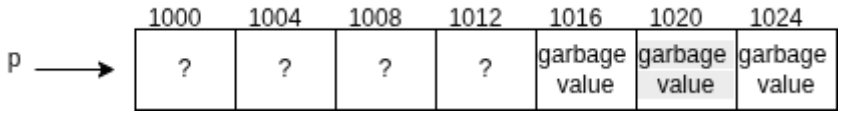
Preserves existing data when expanding.

Syntax: `*ptr = realloc(void* ptr, new-byte-size);`

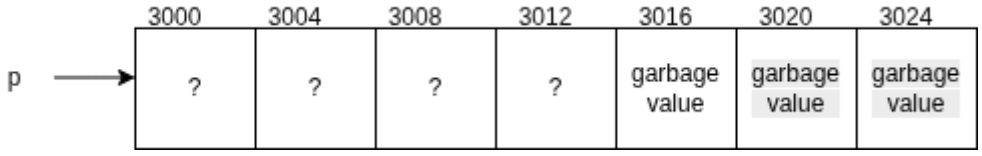
EXAMPLE (start with the **pointer p** after malloc before):

```
int *p = realloc(p, 7 * sizeof(int));
```

1st Case: if sufficent memory is available after address 1012, then the address of p doesn't change



2nd Case: if sufficent memory is not available after address 1012, then the realloc() function allocates memory somewhere else in the head and copies the all content from old memory block to the new memory block. In this case the address of p changes



Memory leak

```
int *p = malloc(100);
p = NULL;
```

Pointer p points to 100 bytes on the heap.

Pointer p changes its address without freeing the memory first (Heap still holds data) --> **It does not crash immediately. Instead, a silent memory leak occurs, which can lead to a crash over time**

Dangling pointer

```
int *p = malloc(sizeof(int));
free(p);
*p = 10;
```

Pointer points to freed memory, but still modifies it → Runtime Error.

Double free

```
free(p);
free(p);
```

→ Runtime error

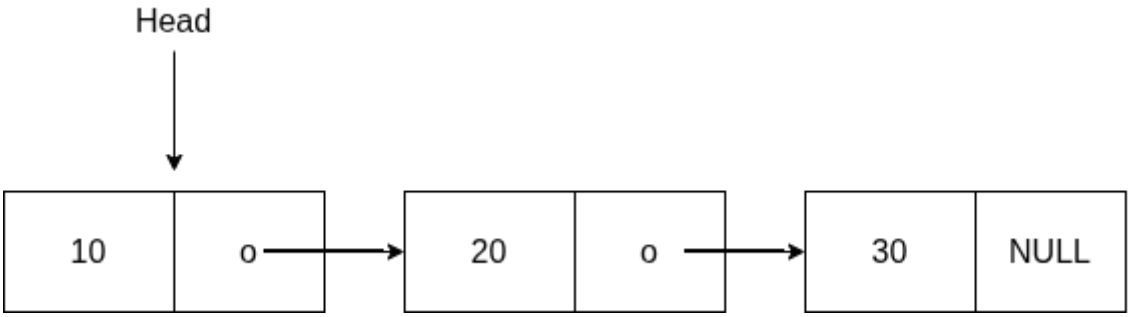
Use-after-free

```
free(p);
printf("%d", *p);
```

→ Runtime error

4. Implement Linked list

Definition of Single Linked List




```
#include <stdio.h>
#include <stdlib.h>
typedef struct Node
{
    int data;
    struct Node *next;
} Node;

Node* create_node(int x) {
```

```
    Node *new_node = malloc(sizeof(Node));

    new_node->data = x;
    new_node->next = NULL;

    return new_node;
}
Node* insert_front(Node **head, int value) {
    Node* new_node = malloc(sizeof(Node));

    new_node->data = value;
    new_node->next = *head;

    *head = new_node;
}
void print_list(Node *head) {
    Node* temp = head;
    while (temp != NULL) {
        printf("%d -> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}
Node* insert_back(Node **tail, int value) {
    Node* new_node = create_node(value);

    Node* temp = *tail;
    while (temp->next != NULL) {
        temp = temp->next;
    }
    temp->next = new_node;
    return new_node;
}
Node* delete_by_value(Node **head, int value) {
    Node* temp = *head;
    Node* prev = NULL;
    while (temp->data != value) {
        prev = temp;
        temp = temp->next;
    }
```

```
    prev->next = temp->next;
    free(temp);
}
Node* search(Node **head, int value) {
    Node* temp = *head;
    while (temp != NULL) {
        if (temp->data == value) {
            return temp;
        }
        temp = temp->next;
    }
    return NULL;
}
int main() {
    Node *head = NULL;
    head = create_node(30);
    head->next =  create_node(50);
    printf("Danh sach lien ket: ");
    print_list(head);

    insert_front(&head, 10);
    print_list(head);

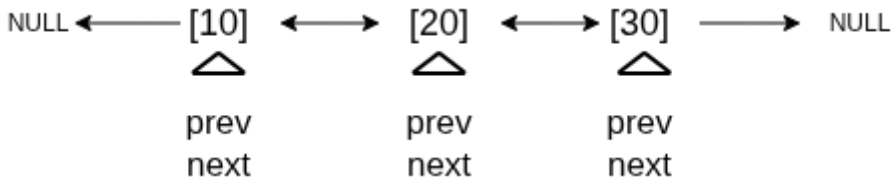
    insert_back(&head, 70);
    print_list(head);

    Node* ketqua = search(&head, 30);
    if (ketqua != NULL) {
        printf("Find the node with value is %d\n", ketqua->data);
    }
    else { printf("Can't find node "); }

    delete_by_value(&head, 50);
    print_list(head);

    free(head->next);
    free(head);
}
```

5. Double Linked List



6. Structure/Typedef/Union

Different of struct và typedef struct

```
struct Person {
    int age;
    char name[32];
};
```

Create struct with **tag name** is Person

-> when declare the variable, must write `struct Person p;`

```
typedef struct {
    int age;
    char name[32];
} Person;
```

`struct` dont have **tag name**

`Person` is a typedef alias **(bí danh)** for this struct

When declare, only need to write `Person p;`

Can combine both 2 method

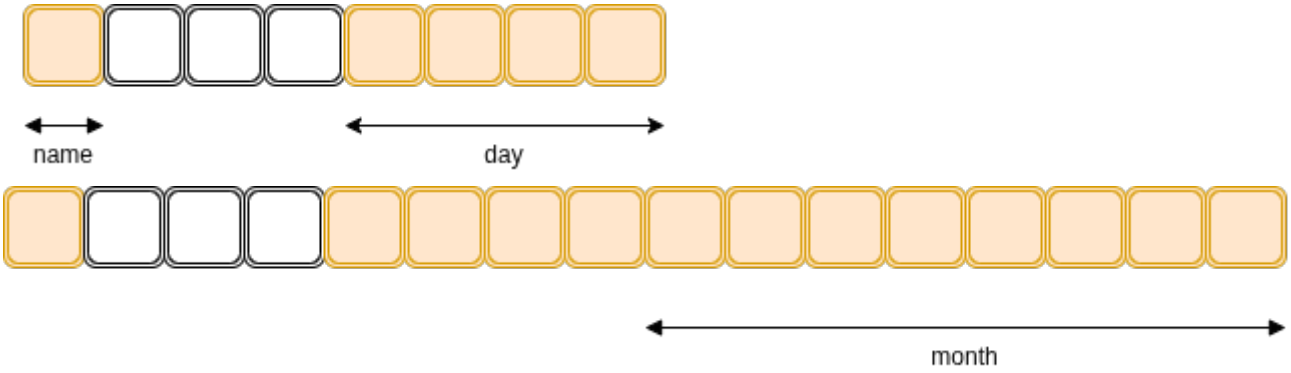
```
typedef struct Person {
    int age;
    char name[32];
} Person;
```

Size of Struct

Example

```
Struct birthday {
  char name;              /* 1 byte */
  int day;               /* 4 byte */
  double month;          /* 8 byte */
};
int main()
{
    printf("%d\n",sizeof(struct birthday));    /* print 16 */
    return 0;
}
```

The compiler will determine that the `month` member of type double is the largest member within the struct, with a size of 8 bytes, then allocates a block sized to fit the largest member and populates the memory areas for the members according to their declaration order within the struct; when a block is exhausted, a new one is allocated.



The purpose of data padding is to ensure that data blocks align with even-numbered byte addresses, thereby increasing the CPU's memory read speed.

STRUCT BY VALUE && STRUCT BY POINTER

Declare 1 struct have typedef alias is Person with 2 attribute

```
typedef struct {
    int age;
    char name[32];
} Person;
```

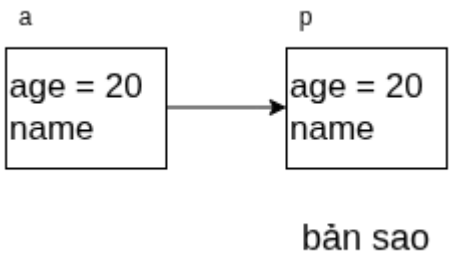
STRUCT BY VALUE

```
void foo(Person p) {
  p.age = 30;
}
```

```
int main() {
  Person a = {20, "Nam"};

  foo(a);

  printf("%d", a.age);  /* 20 */
}
```



fix p is not affect to a

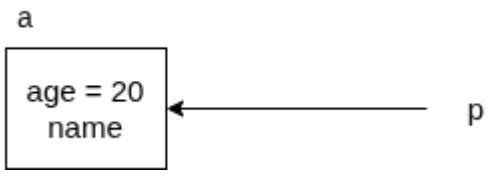
STRUCT BY POINTER

```
void foo(Person *p) {
  p->age = 30;
}

int main() {
  Person a = {20, "Nam"};

  foo(&a);

  printf("%d", a.age);  /* 30 */
}
```



this time, fix p will affect to a --> fix p is fixing a too

7. COMPILATION PIPELINE

7.1 -E stage Compilation

Ex: **compilation.c**

```
/* compilation.c */
#define SIZE 10
int a[SIZE];

#if SIZE >= 10
    printf("large");
#else
    printf("small");
#endif
```

```
Terminal: gcc -E compilation.c -o compilation.i
```

Result (compilation.i)

```
/* compilation.i */
# 0 "compilation.c"                       /* announce: handling compilation.c file */
# 0 "<built-in>"                         /* built-in concept (of compiler) */
# 0 "<command-line>"                      /* preprocessor is handling which is provided by gcc
command */
# 1 "/usr/include/stdc-predef.h" 1 3 4   /* gcc auto include stdc-predef.h (1 3 4: flag of
linemarker) */
# 0 "<command-line>" 2                   /* again, after handle stdc-predef.h */
# 1 "compilation.c"                     /* handle compilation.c, start from line 1 */

int a[10];

    printf("large");
```

compile -E is preprocessing

All macros from `define` are replaced

`#if / #else / #endif` preprocessor check the condition and only keep the right branch

7.2 -S stage compilation

`gcc -S compilation.c -o compilation.s` RESULT:

```
/* compilation.s */
	.file	"compilation.c"
	.text
	.globl	a                /* global variable a */
	.bss
	.align 32
	.type	a, @object
	.size	a, 40           /* a = 40 byte because 10 element * 4 bytes = 40 */
a:
	.zero	40
	.section	.rodata          /* read-only data */
.LC0:
	.string	"large"            /* print "large" */
	.text
	.globl	main
	.type	main, @function
main:                         /* main function */
.LFB0:
	.cfi_startproc
	endbr64
	pushq	%rbp
	.cfi_def_cfa_offset 16
	.cfi_offset 6, -16
	movq	%rsp, %rbp
	.cfi_def_cfa_register 6
	leaq	.LC0(%rip), %rax        /* address of LARGE string */
	movq	%rax, %rdi              /* push it into register rdi */
	movl	$0, %eax
	call	printf@PLT              /* call printf */
	movl	$0, %eax
	popq	%rbp
	.cfi_def_cfa 7, 8
	ret
	.cfi_endproc
.LFE0:
	.size	main, .-main
	.ident	"GCC: (Ubuntu 11.4.0-1ubuntu1~22.04.3) 11.4.0"
	.section	.note.GNU-stack,"",@progbits
	.section	.note.gnu.property,"a"
	.align 8
```

```
 .long 1f - 0f
 .long 4f - 1f
 .long 5
0:
 .string "GNU"
1:
 .align 8
 .long 0xc0000002
 .long 3f - 2f
2:
 .long 0x3
3:
 .align 8
4:
```

GCC do 2 main works, Preprocessor + Compile --> Assembly

7.3 -c stage compilation

`gcc -c main.c -o main.o`

main.c --> Preprocessor --> Compiler --> Assember --> main.o

main.o have machine code but not completely link **-c stage is created OBJECT FILE, stop before LINK**

7.4 Full link

`gcc compilation.o -o compilation`

Linker will combine [main.o + other.o + libraries] --> Linker --> compilation

Result: `large`

8. Function pointers and callback-style APIs

8.1. Function pointers

Pointers store memory addresses. While pointers are commonly used with variables, they can also store the address of a function.

A function pointer is a pointer that stores the address of a function.

- Usage:

Calling functions indirectly

Passing functions as arguments

Implementing callbacks

Creating function tables

8.2 Callback

A callback is the process of passing a function to another function as an argument so that the receiving function can call it later.

```
#include <stdio.h>

int add(int a, int b) { return a + b; }
int subtract(int a, int b) { return a - b; }
int multiply(int a, int b) { return a * b; }
int divide(int a, int b) { return a / b; }

void calculator(int a, int b, int (*callback)(int, int))
{
    printf("%d\n", callback(a, b));
}

int main(void)
{
    int a = 10, b = 2;
    calculator(a, b, add);
    calculator(a, b, subtract);
    calculator(a, b, multiply);
    calculator(a, b, divide);
    return 0;
}
```

Output:

```
12
8
20
5
```

Revision #26
Created 2026-08-19 03:12:26 UTC by Lê Hoài Nam
Updated 2026-08-21 10:19:47 UTC by Lê Hoài Nam